Designing an AI Platform: A Reference Architecture for RAG and Agent-Ready Systems

This document serves both as a reference architecture and as a learning platform for exploring modern AI-native technologies such as RAG frameworks, agent systems, model routing, and LLM observability.

Author: Pau Gonzalez
Date: 03/21/2026
Primary Cloud Platform: Google Cloud Platform (GCP)

---

1. Context **4**
2. Objective **4**
3. High-Level Architecture **5**
Architecture summary 5
Architecture overview 5
Architectural Design Principles 6
Control Flow Overview 6
4. Infrastructure Layer **6**
Purpose 6
Platform Options 7
Rationale 7
5. Data Platform Layer **7**
Purpose 7
Document Processing and Chunking 8
Data Platform Components 8
Technology Options 8
Rationale 8
6. Retrieval and Vector Database Layer **9**
Purpose 9
Technology Options 9
Rationale 9
7. Model Serving / Inference Layer **9**
Purpose 9
Technology Options 10
Rationale 10
8. Model Gateway / Routing Layer **10**
Purpose 10
Technology Options 10

Rationale 11
9. AI Orchestration Layer 11
Purpose 11
Technology Options 11
Agentic Capabilities Included in v1 12
Initial Tool Types for v1 12
Rationale 13
10. Observability and Monitoring Layer 13
Purpose 13
Technology Options 13
Rationale 14
11. Application Layer 14
Purpose 14
Technology Options 14
Rationale 14
12. Cost Considerations 14
Purpose 14
Cost Optimization Principles 15
Rationale 15
13. Security and AI Governance 16
Purpose 16
Governance Controls 16
Rationale 17
14. Reference Architecture Diagrams 18
Layered Platform View 18
RAG Request Flow 19
Control Plane vs Data Plane Diagram 19
15. Evaluation Layer (optional for v1) 20
Purpose 20
Evaluation Methods 20
Example Evaluation Metrics 20
Technology Options 21
16. Implementation Roadmap 21
Phase 1 — Platform Foundation 21
Phase 2 — Data Platform 21
Phase 3 — Retrieval Layer 21
Phase 4 — Model Integration 21
Phase 5 — AI Orchestration 21
Phase 6 — Application Layer 21
Phase 7 — Observability and Evaluation 22
17. Future Architecture Evolution 22
Purpose 22

Is the Architecture Future-Proof? 22
Why the Current Architecture Can Evolve 22
Support for Agentic Architectures 23
Agentic Capabilities That Could Be Added 23
Example Agentic Evolution Path 24
Support for Other Architectural Developments 24
Multi-Model Platforms 24
Hybrid Retrieval 24
Self-Hosted Inference and GPU Infrastructure 25
Stronger Evaluation Loops 25
Enterprise Governance and Policy Engines 25
What Would Need to Change Over Time? 26
Limitations of the Current Version 26
Platform Maturity Roadmap 27
Phase 1 — Prototype 27
Phase 2 — Production AI Application 27
Phase 3 — AI Platform 27
18. Key Architectural Insights 28
19. Additional Architectural Design Principles 28
20. Architecture and Learning Goals Relationship 29
21. Closing Thoughts 30
Appendix A. Technologies to Explore 30
Appendix B. Developer Tooling 30
Rationale 31
Appendix C. AI Applications and Companies Using Similar Architectures 31
Engineering and Platform Copilots 31
Enterprise Knowledge Assistants 32
Customer Support AI 32
Research and Technical Assistants 33
Why This Architecture Works Across Many Use Cases 33

1. Context

Modern AI systems increasingly resemble distributed platforms rather than simple applications.

Capabilities such as retrieval, orchestration, model routing, evaluation, and governance form reusable infrastructure that can support multiple AI products and workflows.

Instead of building isolated AI applications, organizations are beginning to develop AI platforms that provide shared capabilities across many use cases.

These platforms combine components such as:

- large language model inference
- knowledge retrieval systems
- orchestration workflows
- multi-model routing
- evaluation and monitoring
- governance and safety mechanisms

This document presents a reference architecture for an AI platform prototype designed to explore how modern AI-native systems are structured using cloud infrastructure and managed AI services.

The prototype emphasizes platform architecture and system design rather than machine learning model training.

2. Objective

This document defines a reference architecture for a prototype AI platform exploring how modern AI-native systems are structured and deployed.

The prototype demonstrates a reference architecture for building AI applications using modern platform patterns such as retrieval, orchestration, model routing, and managed AI services.

The objective of the prototype is to understand how an AI platform can support multiple potential AI applications. The platform is intentionally designed generically so that different use cases can be implemented on top of the same architectural foundation.

The objectives of the prototype are to understand:

Focus Area	Description
AI application architecture	How modern AI products are structured
Retrieval Augmented Generation	Combining LLMs with knowledge retrieval
Vector databases	Semantic search over documents
Model inference platforms	How LLMs are served in production

Focus Area	Description
AI observability	Monitoring prompts, responses and system health
Platform architecture	Layered architecture patterns for AI systems

3. High-Level Architecture

Architecture summary

The platform follows a layered AI system architecture designed to support Retrieval Augmented Generation (RAG) and future agent-based workflows. The system separates responsibilities across infrastructure, data management, retrieval, model inference, orchestration, and application layers.

At a high level, the platform operates as follows:

Application Layer (API / UI)
↓
AI Orchestration Layer (RAG workflow)
↓
Model Gateway / Routing
↓
Retrieval Layer (vector search)
↓
Model Serving (LLM inference)
↓
Data Platform (document ingestion, chunking, embeddings)
↓
Cloud Infrastructure (GCP)

Architecture overview

The platform is structured into seven logical architecture layers.

Layer	Responsibility
Application Layer	User interface and API access
Observability	Logging, Monitoring
AI Orchestration Layer	RAG workflow orchestration
Model Gateway / Routing Layer	Model selection, policy enforcement and routing
Model Serving Layer	LLM inference and response generation
Retrieval / Vector Database Layer	Semantic search and knowledge retrieval
Data Platform Layer	Document ingestion and embedding pipelines

Layer	Responsibility
Infrastructure Layer	Cloud compute, storage, networking and security

Modern AI systems are increasingly designed as layered platforms rather than single applications. This architectural style mirrors patterns long used in distributed cloud platforms, where responsibilities are separated across layers such as application interfaces, control planes, data services, and infrastructure.

Separating these concerns allows the platform to evolve over time. For example, model providers can change without impacting the application layer, or retrieval techniques can evolve without redesigning orchestration logic.

Architectural Design Principles

Several design principles guide the architecture:

- **Layered Separation of Concerns**
  Each architectural layer focuses on a specific responsibility. This reduces coupling and allows individual components to evolve independently.
- **Platform-First Thinking**
  The architecture is designed to support multiple future AI applications rather than a single product use case.
- **Managed Services First**
  During the prototype phase, managed cloud services are preferred to reduce operational complexity and accelerate experimentation.
- **Future Evolution**
  The architecture is intentionally designed so that components such as model inference or retrieval infrastructure can be replaced or upgraded without redesigning the entire platform.

Control Flow Overview

A simplified request flow through the system is:

User Request
→ Application Layer
→ AI Orchestration Layer
→ Model Gateway / Routing
→ Retrieval Layer (if needed)
→ Model Inference
→ Response Returned to User

This separation mirrors patterns seen in modern AI products such as developer assistants, enterprise knowledge assistants, and AI copilots.

4. Infrastructure Layer

Purpose

Provides the foundational platform for:

- compute
- networking
- storage
- identity
- security

All higher layers of the AI platform depend on this layer.

Platform Options

Platform	Example Services	Advantages	Tradeoffs
Google Cloud Platform (Selected)	GKE, Cloud Run, Vertex AI, Cloud Storage	Strong ML ecosystem, Kubernetes native	Smaller enterprise footprint than Azure
Microsoft Azure	Azure OpenAI, AKS, Azure ML	Enterprise AI adoption	Less common in AI-native startups
AI-Native GPU Clouds	CoreWeave, Lambda	High-performance AI workloads	Limited general cloud ecosystem

Rationale

Decision Factor	Explanation
AI ecosystem maturity	GCP has strong native tooling for AI and ML workloads
Kubernetes leadership	GKE is widely used by AI startups and ML platforms
Startup ecosystem alignment	Many Silicon Valley AI companies operate on GCP infrastructure
Managed + custom flexibility	Supports both managed AI services and custom infrastructure

5. Data Platform Layer

Purpose

Responsible for managing the knowledge base used by the AI system.

Key responsibilities include:

- storing raw documents
- ingestion pipelines
- embedding generation
- metadata management

Document Processing and Chunking

Documents uploaded to Cloud Storage are processed by the ingestion pipeline, where text is extracted and divided into smaller chunks before embeddings are generated. Chunking helps ensure that retrieved content remains relevant and within model context limits.

Each chunk retains metadata such as the source document and location within the document to support traceability and accurate retrieval. Embeddings are then generated for each chunk and stored in the vector database for semantic search.

Data Platform Components

Component	Responsibility
Document Storage	Stores PDFs, documentation and internal knowledge artifacts
Ingestion Pipeline	Processes documents into structured format
Embedding Pipeline	Converts text to vector embeddings
Metadata Store	Tracks document metadata and indexing
Document Processing / Chunking	Splits extracted text into retrieval-ready chunks with metadata

Technology Options

Option	Technologies	Advantages	Tradeoffs
Cloud Storage Pipeline (Selected)	Cloud Storage + Cloud Run ingestion service + Vertex AI embeddings	Simple architecture, low operational overhead	Limited analytics capabilities
Data Warehouse	BigQuery, Snowflake	Advanced analytics integration	Higher complexity
Data Lake Architecture	Lakehouse platforms	Flexible data processing	Operational complexity

Rationale

Decision Factor	Explanation
Simplicity	Minimal infrastructure required for document storage
Scalability	Object storage easily scales for large document sets
Integration	Works seamlessly with embedding pipelines

6. Retrieval and Vector Database Layer

Purpose

Provides semantic search capabilities.

Documents are converted to embeddings and stored in a vector database.

When a user query arrives:

- Query converted to embedding
- Similar embeddings retrieved
- Relevant documents passed to the LLM

Technology Options

Technology	Type	Advantages	Tradeoffs
Vertex AI Vector Search (Selected)	Managed	Native GCP integration, scalable	Less infrastructure control
Pinecone	SaaS	Very simple deployment	Vendor dependency
Weaviate	Open source	Flexible deployment	Requires infrastructure management
Milvus	Open source	High performance vector search	Operational complexity

Rationale

Decision Factor	Explanation
Managed service	Reduces operational overhead
Native integration	Works seamlessly with other GCP services
Scalability	Designed for large scale vector indexing

7. Model Serving / Inference Layer

Purpose

Executes large language models responsible for generating responses.

This layer receives prompts from the orchestration layer and produces the final output.

Technology Options

Option	Technologies	Advantages	Tradeoffs
Managed Model APIs (Selected)	Vertex AI, OpenAI, Anthropic	Quick deployment, minimal infrastructure	External API dependency
Self-Hosted Inference	vLLM, NVIDIA Triton, Ray Serve	Control over infrastructure and cost	Requires GPU infrastructure
Hybrid Architecture	Managed + self-hosted models	Flexibility	Higher architectural complexity

Rationale

Decision Factor	Explanation
Speed of experimentation	Rapid prototype development
Reliability	Managed infrastructure scaling
Simplicity	No GPU infrastructure management required

8. Model Gateway / Routing Layer

Purpose

The model gateway layer provides a control point between orchestration and inference. It is responsible for selecting the right model, applying policy checks, enforcing routing rules, and enabling fallback behavior.

This layer becomes increasingly important as AI systems begin using multiple models with different cost, latency, and capability profiles.

Technology Options

Option	Technologies	Advantages	Tradeoffs
Lightweight Gateway Logic (Selected)	Custom routing logic in Python service	Simple to implement, good for learning	Limited standardization
Model Gateway Libraries / Proxies	LiteLLM-style routing, API proxies	Easier multi-model abstraction	Adds another dependency layer
Enterprise AI Gateway	Internal platform gateway	Centralized governance and policy control	Higher complexity

Rationale

Decision Factor	Explanation
Industry relevance	Multi-model routing is increasingly common in AI-native systems
Learning value	Helps understand cost, latency, and capability tradeoffs
Scalability	Creates a clean path toward future platformization
Simplicity	Lightweight logic is sufficient for the prototype

9. AI Orchestration Layer

Purpose

Coordinates the AI workflow responsible for connecting retrieval, model inference, and application requests.

Key responsibilities include:

- retrieving relevant documents from the vector database
- constructing prompts using retrieved context
- invoking the LLM inference layer
- formatting and returning responses
- coordinating multi-step workflows or tool usage
- managing lightweight planner / executor flows for bounded agentic tasks

This layer effectively acts as the control plane of the AI system, managing how information flows between the application, retrieval system, models, and future tool-using workflows.

Technology Options

Option	Technologies	Advantages	Tradeoffs
Custom Orchestration Service + Framework Exploration (Selected)	Python service, with optional LangChain / LlamaIndex / CrewAI / AutoGPT modules	Keeps core architecture transparent while enabling experimentation	Slightly broader implementation scope
RAG Framework-Centric Approach	LangChain, LlamaIndex	Faster development and strong ecosystem	Additional abstraction and framework dependency
Agent-First Approach	CrewAI, AutoGPT	Good for multi-step reasoning and autonomous task workflows	Less predictable behavior and higher complexity

Agentic Capabilities Included in v1

Capability	Description	Scope Boundary
Query classification	Distinguish between direct Q&A, deeper retrieval, and tool-assisted tasks	No autonomous task creation
Tool registry	Small set of explicitly allowed tools available to orchestration	Only approved read-only tools
Planner / executor flow	Lightweight step planning before retrieval or tool use	Short, bounded workflows only
Second-pass retrieval	Perform an additional retrieval step if first-pass context is insufficient	Limited to a small number of steps
Human-in-the-loop checkpoint	Return proposed actions or next steps for user confirmation	No autonomous external action execution

Initial Tool Types for v1

Tool Type	Example Purpose
Retrieval tool	Retrieve additional documentation or architecture notes
Metadata lookup tool	Inspect document metadata or source type
Structured comparison tool	Compare architectures, services, or tradeoffs
Checklist tool	Return deployment or troubleshooting checklists

On top of that base, selected frameworks can be evaluated in isolated experiments:

Framework	Exploration Purpose
LlamaIndex	Explore document indexing and retrieval-centric RAG patterns
LangChain	Explore orchestration abstractions, tool integrations, and chains
CrewAI	Explore multi-agent coordination patterns
AutoGPT	Explore autonomous task execution and agent loops

This allows the prototype to remain architecturally clear while also functioning as a learning platform for AI-native orchestration frameworks and basic agentic workflows.

Rationale

Decision Factor	Explanation
Architectural clarity	The custom orchestration layer keeps the core RAG workflow explicit and easy to understand
Exploration flexibility	Frameworks can be tested without redesigning the surrounding platform
Comparative learning	Enables side-by-side comparison between manual implementation and framework-driven approaches
Risk control	Keeps experimental agent frameworks isolated from the core reference architecture
Practical agentic learning	Introduces tool use and planning without requiring a full autonomous agent runtime

10. Observability and Monitoring Layer

Purpose

Ensures the AI system is:

- observable
- measurable
- auditable

Observability becomes increasingly important as AI systems move into production.

Technology Options

Option	Technologies	Advantages	Tradeoffs
Cloud Native Observability (Selected)	Cloud Logging, Monitoring	Simple and integrated	Limited AI-specific metrics
AI Observability Platforms	LangSmith, Arize	Advanced prompt tracing and evaluation	Additional cost
Custom Evaluation Pipelines	Internal evaluation tooling	Full control	Engineering overhead

Rationale

Decision Factor	Explanation
Simplicity	Minimal operational overhead

Decision Factor	Explanation
Integration	Native integration with GCP services
Prototype scope	Sufficient for early experimentation

11. Application Layer

Purpose

Provides the interface used by users to interact with the AI system.

Responsibilities include:

- receiving user queries
- authenticating users
- routing requests to the orchestration service

Technology Options

Option	Technologies	Advantages	Tradeoffs
Web Application + API (Selected)	Cloud Run service	Simple deployment model	Basic user interface
Chatbot Integration	Slack / Teams bots	Integrated enterprise workflows	Limited UI control
Developer Platform UI	Internal developer portal	Rich experience	Higher development effort

Rationale

Decision Factor	Explanation
Simplicity	Minimal UI required for prototype
Flexibility	API can be reused by other applications
Deployment speed	Easily deployed using Cloud Run

12. Cost Considerations

Purpose

This section identifies the primary cost drivers of the prototype and the main optimization levers.

Layer	Primary Cost Driver	Cost Risk	Optimization Strategy
Application Layer	Request volume	Low	Use Cloud Run autoscaling and

Layer	Primary Cost Driver	Cost Risk	Optimization Strategy
			request-based compute
AI Orchestration Layer	Processing per request	Low	Keep orchestration logic lightweight
Model Serving / Inference	Token usage and model choice	High	Use smaller models where appropriate, caching, prompt optimization
Retrieval / Vector DB	Index size and query volume	Medium	Tune chunking strategy and embedding dimensions
Data Platform	Storage and embedding generation	Medium	Batch ingestion and lifecycle management
Infrastructure Layer	Network egress and managed services	Medium	Keep services within the same cloud region

Cost Optimization Principles

Principle	Description
Start managed	Reduce time-to-value and avoid early infrastructure overhead
Optimize inference first	Inference is usually the largest cost center in GenAI systems
Right-size retrieval	Poor chunking and oversized indexes increase cost without improving quality
Measure before optimizing	Track usage patterns before introducing self-hosted infrastructure
Delay GPU ownership	Managed APIs are more cost-effective during the prototype phase

Rationale

Decision Factor	Explanation
Prototype economics	Managed services reduce upfront engineering cost
Fast iteration	Focus on architecture validation instead of platform tuning

Decision Factor	Explanation
Visibility	Cost can be attributed by layer and used later in architecture decisions

13. Security and AI Governance

Purpose

This section outlines the core security and governance considerations for the prototype, especially for enterprise-style AI systems handling internal knowledge.

Risk Area	Example Risk	Mitigation Strategy
Prompt injection	Malicious content manipulates model behavior	Input filtering, prompt isolation, retrieval controls
Data leakage	Sensitive documents exposed in responses	Access control, document classification, response filtering
PII exposure	Personal data appears in retrieved context	Data minimization, redaction, document tagging
Unauthorized access	Users query documents outside their access scope	IAM, service identity, authorization checks
Hallucinations	Model generates unsupported answers	Retrieval grounding, answer citation, evaluation pipeline
Abuse / misuse	Excessive or unsafe usage patterns	Logging, rate limiting, auditability
Unsafe tool usage	Agentic workflow attempts unsupported actions	Tool allowlist, read-only initial tools, explicit approval checkpoints

Governance Controls

Control Area	Prototype Approach	Future Production Evolution
Identity	Cloud-native IAM and service accounts	Fine-grained RBAC and document-level authorization
Secrets	Managed secret storage	Centralized enterprise secrets management
Auditability	Cloud logs and request tracing	Full prompt/response audit trails
Data handling	Controlled document set	Data classification and policy enforcement

Control Area	Prototype Approach	Future Production Evolution
Model risk	Manual evaluation	Automated evaluation and guardrails
Tool permissions	Small approved tool registry	Policy-driven tool authorization and action controls
Approval flow	Human confirmation for proposed actions	Role-based approval workflows and action governance

Rationale

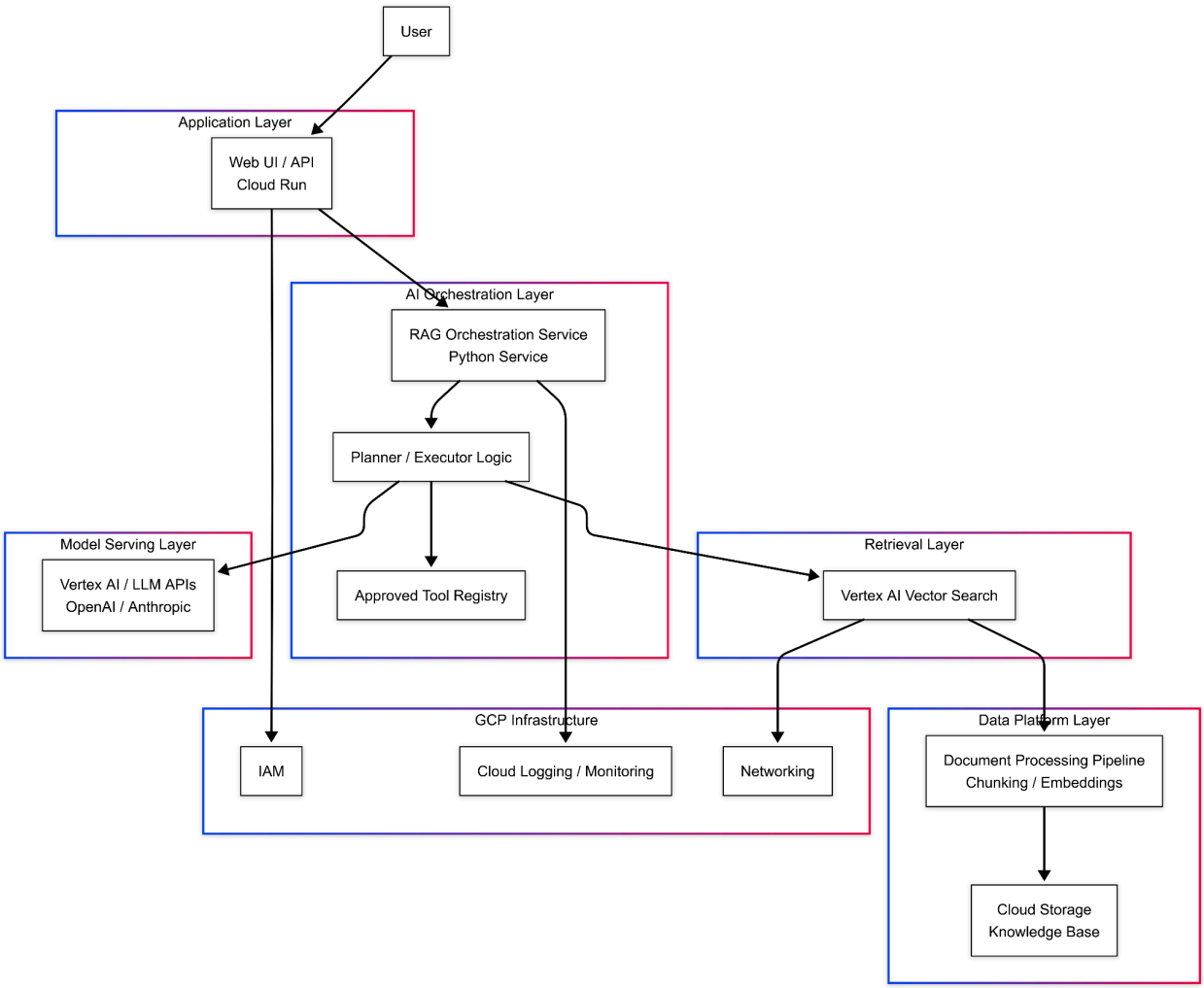
Decision Factor	Explanation
Enterprise relevance	Security and governance are essential for credible AI platform design
Leadership signal	Demonstrates platform thinking beyond simple prototypes
Scalability	Early governance patterns make future productionization easier
Agentic readiness	Bounded tool permissions create a safe path toward agentic evolution

14. Reference Architecture Diagrams

The following diagrams illustrate the structural architecture of the platform and the runtime execution flow for user queries.

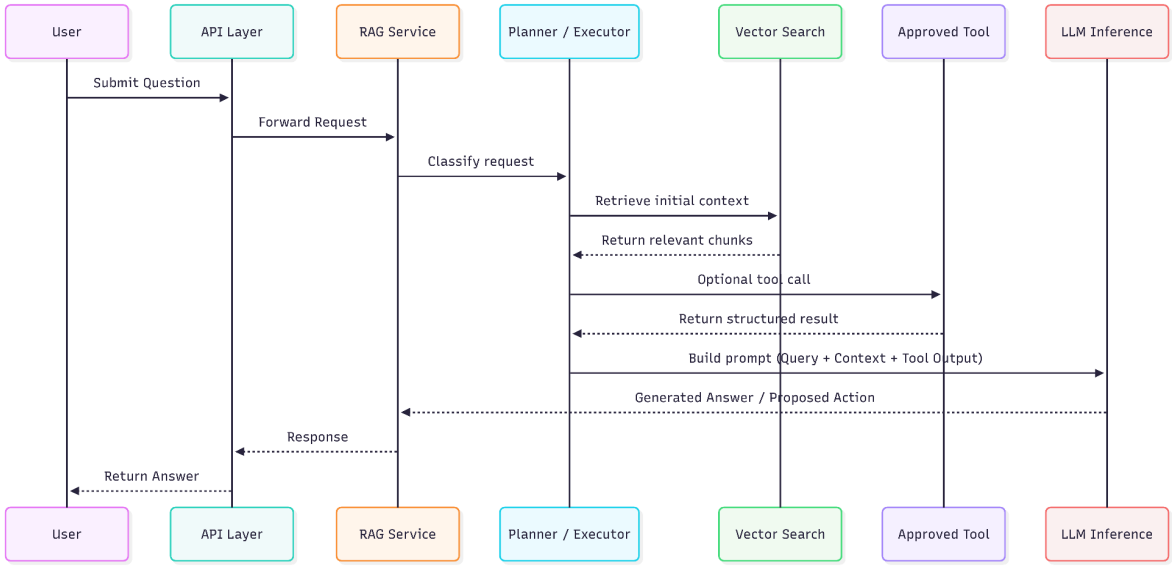
Layered Platform View

The following diagram represents the reference architecture of the prototype AI platform. It illustrates the relationship between the application layer, orchestration services, retrieval components, model inference, and the underlying cloud infrastructure.



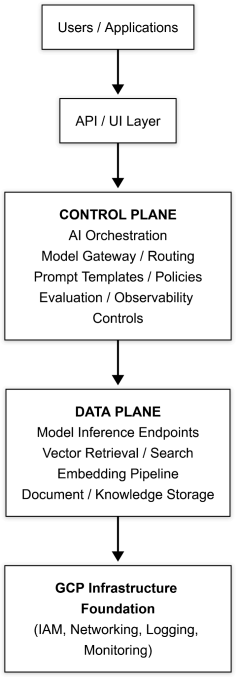
RAG Request Flow

The following diagram shows the runtime execution path when a user sends a query to the system.



Control Plane vs Data Plane Diagram

The following diagram separates the platform into control plane and data plane responsibilities, which is a common way AI-native platform teams reason about architecture.



Why This Distinction Matters

Plane	Responsibility
Control Plane	Decides how requests are handled, which models are used, how policies are applied, and how system behavior is monitored
Data Plane	Executes the actual work of retrieval, inference, embedding generation, and data access

15. Evaluation Layer (optional for v1)

Purpose

The evaluation layer measures the quality, safety, and accuracy of responses generated by the AI system. Modern AI platforms rely on continuous evaluation loops to monitor model performance and improve system behavior.

Evaluation Methods

Method	Description	Example Use
Human evaluation	Manual review of responses	Early prototype validation
LLM-based evaluation	Models score responses automatically	Response quality checks
Benchmark datasets	Predefined test queries	Regression testing
Retrieval evaluation	Measure relevance of retrieved documents	RAG tuning
Tool-use evaluation	Measure correctness of tool selection and outputs	Basic agentic workflow validation

Example Evaluation Metrics

Metric	Purpose
Answer accuracy	Measures correctness of responses
Retrieval relevance	Determines if correct documents are retrieved
Hallucination rate	Tracks unsupported claims
Latency	Measures user experience
Tool invocation accuracy	Measures whether the right tool was selected
Workflow success rate	Measures whether bounded multi-step workflows reach useful outcomes

Technology Options

Tool	Category	Purpose
LangSmith	LLM observability	Prompt tracing and evaluation
Arize	Model monitoring	Performance monitoring
WhyLabs	Data monitoring	Detect drift and anomalies

16. Implementation Roadmap

The architecture can be implemented incrementally through the following phases:

Phase 1 — Platform Foundation

- GCP project setup
- IAM and service identities
- networking and security baseline
- Terraform infrastructure setup

Phase 2 — Data Platform

- document ingestion pipeline
- chunking and embedding generation
- knowledge base storage

Phase 3 — Retrieval Layer

- vector index creation
- semantic search queries
- retrieval evaluation

Phase 4 — Model Integration

- managed model APIs
- inference abstraction layer
- model routing logic

Phase 5 — AI Orchestration

- RAG workflow orchestration
- tool registry and bounded agent patterns

Phase 6 — Application Layer

- minimal web interface and API
- user interaction flow

Phase 7 — Observability and Evaluation

- logging and tracing
- response evaluation
- governance controls

This roadmap illustrates how the architectural components described in this document can be built incrementally to form a working AI platform.

17. Future Architecture Evolution

Purpose

This section evaluates whether the reference architecture is designed to remain useful as AI systems evolve beyond basic Retrieval Augmented Generation (RAG) patterns into more advanced forms such as agentic workflows, multi-model systems, tool-using assistants, and future platform capabilities.

The architecture is intentionally structured in logical layers so that new capabilities can be introduced without requiring a full redesign of the platform.

Is the Architecture Future-Proof?

At a high level, yes. The architecture is future-oriented because it separates core concerns into modular layers:

- application access
- orchestration and control logic
- model routing
- model inference
- retrieval and knowledge access
- data ingestion and storage
- infrastructure and governance

This separation makes it possible to evolve the system incrementally as AI capabilities and architectural patterns change.

The design is not tied to a single model, a single retrieval implementation, or a single workflow pattern. Instead, it provides a foundation that can support multiple future architectural directions.

Why the Current Architecture Can Evolve

Architectural Characteristic	Why It Supports Future Evolution
Layered design	Allows new capabilities to be added without redesigning the entire system
Explicit orchestration layer	Provides a place to introduce agent workflows, tool invocation, and multi-step reasoning

Architectural Characteristic	Why It Supports Future Evolution
Model gateway layer	Enables multi-model routing, fallback logic, and future policy enforcement
Managed model APIs	Makes it easy to compare and swap model providers
Retrieval layer abstraction	Supports future expansion from simple vector retrieval to hybrid or graph-based retrieval
Evaluation and governance sections	Provide a path to production-grade controls as capabilities expand

Support for Agentic Architectures

One of the most important future architectural developments is the move from single-turn question answering systems to agentic systems that can reason over multiple steps, use tools, and potentially take actions.

This architecture can support agentic evolution because the key prerequisites already exist:

Existing Layer	Agentic Relevance
AI Orchestration Layer	Can coordinate multi-step planning and execution
Model Gateway / Routing Layer	Can choose models optimized for planning, acting, or summarization
Retrieval Layer	Provides context grounding for agents
Application Layer	Can expose task-oriented workflows rather than only chat interactions
Evaluation Layer	Can measure success of multi-step workflows
Security / Governance Layer	Can limit unsafe or unauthorized tool usage

Agentic Capabilities That Could Be Added

Agentic Capability	How It Fits the Architecture
Multi-step reasoning	Implemented within the orchestration layer
Tool use	Added as callable tools managed by orchestration
Planner / executor patterns	Added as orchestration sub-components
Multi-agent collaboration	Introduced as additional orchestration modules
Human-in-the-loop approval	Exposed through the application and governance layers

Agentic Capability	How It Fits the Architecture
Autonomous remediation	Added cautiously with policy checks and approval controls

Example Agentic Evolution Path

Phase	Evolution
Current state	Retrieve context, use bounded tools when needed, and generate grounded answer
Next step	Retrieve context and invoke tools for additional data
Later step	Plan and execute multi-step workflows
Advanced state	Coordinate multiple specialized agents with governance controls

This means the architecture can evolve from RAG assistant → tool-using copilot → agentic workflow system without changing its basic structure.

Support for Other Architectural Developments

The architecture is also compatible with several other likely developments in AI platform design.

Multi-Model Platforms

Future AI platforms will rarely rely on a single model. Different models may be used for:

- reasoning
- summarization
- retrieval augmentation
- cost-sensitive requests
- domain-specific tasks

The current Model Gateway / Routing Layer is already designed for this.

Future Development	Current Architectural Support
Best-model selection	Model gateway and routing policies
Fallback models	Gateway retry and failover logic
Cost-aware routing	Gateway policy rules
Specialized domain models	Routing logic and orchestration

Hybrid Retrieval

The architecture can evolve beyond pure vector retrieval.

Possible future retrieval patterns include:

- hybrid keyword + vector search
- graph retrieval

- structured data lookup
- knowledge graph augmentation
- memory layers for agent systems

The existing Retrieval and Vector Database Layer can be expanded to support these without changing the rest of the platform.

Self-Hosted Inference and GPU Infrastructure

As workloads grow, platforms often move from managed APIs toward self-hosted inference.

The current architecture can support that transition because the Model Serving Layer is already abstracted from the application and orchestration layers.

Deployment Mode	Architectural Impact
Managed model APIs	Current selected approach
Self-hosted inference	Change inside Model Serving Layer
Hybrid inference	Coordinated through Model Gateway

This means the platform can adopt:

- vLLM
- NVIDIA Triton
- Ray Serve
- dedicated GPU infrastructure

without redesigning the application or retrieval layers.

Stronger Evaluation Loops

Future AI platforms increasingly depend on continuous evaluation.

The architecture already includes a dedicated Evaluation Layer, which creates a path to:

- benchmark suites
- automated regression testing
- retrieval quality scoring
- hallucination detection
- agent task success evaluation

This becomes especially important in agentic systems where correctness depends on multiple steps rather than one answer.

Enterprise Governance and Policy Engines

As AI systems become more autonomous, governance becomes more important.

The current architecture can evolve to support:

- policy-driven tool permissions
- document-level authorization
- response filtering
- approval workflows
- audit trails for agent actions

This is enabled by the existing combination of:

- model gateway
- orchestration layer
- security and governance controls
- observability and evaluation layers

What Would Need to Change Over Time?

Although the architecture is future-ready, some layers would need to become more sophisticated as the platform matures.

Layer	Likely Future Change
Application Layer	Move from simple chat/API to workflow-oriented interfaces
Orchestration Layer	Add planners, tool registries, memory, and agent coordination
Model Gateway	Add richer routing, quotas, safety policies, and provider abstraction
Retrieval Layer	Support hybrid retrieval, graph retrieval, and session memory
Evaluation Layer	Add automated scoring and continuous regression pipelines
Governance Layer	Add approvals, action boundaries, and stronger auditability

These are evolutionary changes, not signs that the current architecture is incorrect.

Limitations of the Current Version

The current reference architecture is future-proof in structure, but not yet future-complete in implementation.

Current limitations include:

Limitation	Why It Matters
Lightweight orchestration	Sufficient for prototype, but not full agent runtime
Managed model APIs only	Limits learning about deep inference infrastructure
Vector retrieval focus	Does not yet include broader memory or graph retrieval patterns
Basic observability	Needs richer tracing for complex agent workflows
No tool registry yet	Tool invocation patterns would need to be added

These limitations are acceptable for a prototype because the goal is to create a stable architectural base that can evolve later.

Platform Maturity Roadmap

This section describes how the prototype architecture could evolve into a production-grade AI platform.

Phase 1 — Prototype

Focus: learning and experimentation.

Goal: Understand architecture patterns and validate use cases.

Characteristic	Implementation
Models	Managed APIs (Vertex AI, OpenAI, Anthropic)
Retrieval	Managed vector database
Orchestration	Lightweight Python service with bounded tool use and planner / executor flow
Evaluation	Manual testing

Phase 2 — Production AI Application

Focus: reliability and operational maturity.

Goal: Operate a stable AI application serving real users.

Capability	Evolution
Evaluation	Automated evaluation pipelines
Observability	Prompt tracing and monitoring
Security	Fine-grained access control
Scaling	Traffic-based autoscaling

Phase 3 — AI Platform

Focus: enabling multiple teams to build AI applications.

Goal: Transform the system from a single AI application into a reusable enterprise AI platform.

Platform Capability	Description
Model registry	Manage multiple model versions
Multi-model routing	Select models dynamically
Shared retrieval infrastructure	Centralized knowledge systems
Evaluation platform	Continuous model benchmarking
Governance framework	Enterprise AI policy enforcement

18. Key Architectural Insights

Insight	Explanation
AI systems resemble distributed cloud platforms	Similar architecture patterns to microservices
Vector databases are core infrastructure	Enable semantic retrieval for LLMs
Model inference introduces GPU scaling challenges	Requires specialized infrastructure
Orchestration becomes the control plane	Determines how AI applications behave
Observability is essential	AI responses must be monitored and evaluated

19. Additional Architectural Design Principles

The following design principles further guide how this architecture should evolve as the prototype is implemented and potentially expanded into a production platform.

Design Principle	Description	Architectural Impact
Platform-First Thinking	Design infrastructure that supports multiple AI applications	Avoid hard-coding workflows for a single use case
Loose Coupling	Each layer should interact through well-defined interfaces	Allows replacing models, retrieval engines, or orchestration frameworks independently
Observability by Default	Logging, tracing, and evaluation should be built into the platform from the beginning	Enables debugging and model quality tracking
Security by Design	Access control and governance controls should be embedded in architecture rather than added later	Prevents data leakage and unsafe model usage

Incremental Evolution	The system should evolve gradually from prototype → production → platform	Reduces architectural rework and risk

These principles mirror patterns used in modern cloud platform design and are especially important for AI systems that integrate multiple rapidly evolving technologies.

20. Architecture and Learning Goals Relationship

This reference architecture is intentionally designed not only as a blueprint for a prototype AI system but also as a learning platform for understanding modern AI infrastructure.

Learning Dimension	Architectural Component	Outcome
Cloud infrastructure	Infrastructure Layer	Understand how AI systems run on modern cloud platforms
Knowledge systems	Data Platform + Retrieval Layers	Learn how documents become searchable knowledge
LLM integration	Model Serving Layer	Understand inference APIs and model behavior
Workflow design	Orchestration Layer	Learn how AI applications coordinate models, tools, and data
Platform thinking	Control Plane vs Data Plane	Understand reusable AI platform architecture

By exploring each layer individually, engineers can gain a deeper understanding of how modern AI systems are designed and operated.

This educational perspective is one of the key motivations behind developing the prototype architecture.

21. Closing Thoughts

This reference architecture provides a structured way to reason about modern AI platform design.

By separating orchestration, model routing, retrieval, inference, and governance into modular layers, the architecture provides a foundation capable of evolving toward more advanced AI systems and agentic workflows.

Appendix A. Technologies to Explore

Layer	Technologies to Explore	Learning Goal
Application Layer	Cloud Run, simple web UI, API service	Understand how AI apps are exposed to users
AI Orchestration Layer	Custom Python service, LangChain, LlamaIndex	Compare manual RAG orchestration vs frameworks
Agent Patterns	CrewAI, AutoGPT	Explore multi-step reasoning and tool use
Model Gateway / Routing	LiteLLM-style routing patterns, fallback logic, policy checks	Understand multi-model control and abstraction
Model Serving	Vertex AI, OpenAI, Anthropic	Compare model capabilities, latency and cost
Retrieval Layer	Vertex AI Vector Search, hybrid retrieval patterns	Learn semantic search and relevance tradeoffs
Evaluation / Observability	LangSmith, Arize, WhyLabs	Understand evaluation, tracing and AI monitoring
Infrastructure Layer	GCP, GKE concepts, GPU cloud awareness	Learn how AI services map onto platform infrastructure

Appendix B. Developer Tooling

The following tools support development of the platform but are not part of the runtime architecture. They accelerate implementation, experimentation, and iteration while the deployed system runs on GCP.

Tool Category	Selected Tools	Purpose
AI Coding Assistant	Claude Code, GitHub Copilot	Accelerate coding, refactoring, and experimentation during platform development

Tool Category	Selected Tools	Purpose
IDE	VS Code or Cursor	Primary development environment
Version Control	Git + GitHub	Source control and collaboration
Containerization	Docker	Package services for Cloud Run deployment
Local Testing	Python virtual environment / uv	Dependency management and local testing
Diagramming	Mermaid, draw.io	Architecture diagrams and documentation

Rationale

Decision Factor	Explanation
Development velocity	AI coding assistants significantly accelerate prototyping
Platform independence	Developer tools remain separate from the runtime architecture
Reproducibility	Git and containerization ensure reproducible deployments
Documentation clarity	Diagram tools support architecture communication

These tools are part of the developer workflow, while the runtime architecture described earlier is deployed on GCP services such as Cloud Run, Vertex AI, Cloud Storage, and Vertex AI Vector Search.

Appendix C. AI Applications and Companies Using Similar Architectures

Although this document focuses on a reference architecture, similar architectural patterns are widely used across modern AI-native products. These systems typically combine retrieval, orchestration, model routing, and inference in a layered platform design.

Engineering and Platform Copilots

Company	Product Pattern	Description
Datadog	AI Observability Assistant	Helps engineers investigate logs, metrics, and incidents using AI-powered explanations
PagerDuty	Incident Response Copilot	Assists engineers during outages using

Company	Product Pattern	Description
		operational runbooks and incident data
Sourcegraph	Developer / Code Assistant	Helps developers understand codebases, systems, and engineering documentation
Backstage Ecosystem	Developer Platform Assistants	Provide internal developer platform guidance and documentation retrieval

These systems typically combine:

- retrieval over engineering documentation
- operational runbooks
- system diagnostics
- developer workflow assistance

Enterprise Knowledge Assistants

Company	Product Pattern	Description
Glean	Enterprise Knowledge Assistant	Retrieves and synthesizes information from internal enterprise systems
Notion	Workspace Knowledge Assistant	Answers questions across internal documentation and knowledge bases
Atlassian	AI for Jira / Confluence	Retrieves project documentation and summarizes organizational knowledge

These applications primarily rely on RAG architectures over internal knowledge bases.

Customer Support AI

Company	Product Pattern	Description
Intercom	AI Support Assistant	Automates responses using product documentation and customer conversations
Zendesk	AI Support Automation	Uses knowledge bases and ticket history to generate support responses

Company	Product Pattern	Description
Salesforce	AI Service Agents	AI assistants integrated into enterprise customer service workflows

These systems combine:

- retrieval over support documentation
- structured customer or ticket data
- workflow orchestration

Research and Technical Assistants

Company	Product Pattern	Description
Perplexity	AI Research Assistant	Retrieves information from external sources and synthesizes answers
OpenAI	Knowledge Assistants and Copilots	AI assistants built around retrieval and reasoning over documents
Anthropic	Claude Enterprise Assistants	AI assistants integrated into enterprise workflows

Why This Architecture Works Across Many Use Cases

Despite differences in product experience, most AI-native systems share the same underlying architectural components:

Common Pattern	Role in Architecture
Retrieval	Provides grounded knowledge context
Orchestration	Coordinates prompts, tools, and models
Model inference	Generates responses or actions
Data platform	Manages documents and knowledge sources
Infrastructure	Provides scalable compute and storage

Because these patterns appear across many industries, the AI Platform Reference Architecture can support a wide range of applications beyond any single product use case.